

I. THE NEED FOR ORCHESTRATION

In a real-world Big Data environment, you don't just have one Python script. You have hundreds. Some must run every hour, others only after a specific database update finishes. If a script fails at 3:00 AM, you need a system that automatically retries it and alerts the engineer.

Apache Airflow is an open-source platform used to programmatically author, schedule, and monitor workflows. It allows you to treat "Workflows as Code."

1.1. Why not use Cron Jobs?

While Cron is a simple Linux scheduler, it lacks:

- **Dependency Management:** "Run Task B only if Task A succeeds."
- **Visibility:** A UI to see what failed and why.
- **Error Handling:** Automated retries with exponential backoff.
- **Scalability:** Distributing tasks across a cluster of workers.

II. CORE CONCEPTS: THE DAG

The heart of Airflow is the **DAG (Directed Acyclic Graph)**.

- **Directed:** The workflow has a specific direction (Start $\rightarrow$ End).
- **Acyclic:** There are no loops. A task cannot go back to a previous task.
- **Graph:** A collection of nodes (Tasks) and edges (Dependencies).

2.1. Operators: The Building Blocks

An Operator defines a single unit of work. Common operators include:

- **PythonOperator:** Executes a Python function.
- **BashOperator:** Runs a shell command.
- **SQLExecuteQueryOperator:** Runs a query in a database (Snowflake, Postgres, etc.).
- **Sensor:** A special operator that "waits" for something to happen (e.g., a file to appear in S3).

III. AIRFLOW ARCHITECTURE: BEYOND THE SURFACE

Airflow is composed of several components working together to ensure high availability:

1. **Web Server:** The Flask-based UI where you trigger and monitor DAGs.
2. **Scheduler:** The "Brain" that monitors all tasks and triggers them when dependencies are met.
3. **Executor:** Determines *how* the work gets done (Sequential, Local, or Celery for distributed work).
4. **Worker:** The process that actually executes the logic of the task.
5. **Metadata Database:** Usually PostgreSQL, storing the state of all tasks and DAGs.

IV. WORKFLOW LIFECYCLE & TASK STATES

When a DAG runs, each task goes through several states:

- **Scheduled:** The Scheduler has decided the task should run.

- **Queued:** The task is waiting for a worker to become available.
- **Running:** The task is currently executing.
- **Success/Failed:** The final outcome.
- **Upstream_failed:** The task didn't run because its parent task failed.

V. PRACTICAL LAB: CREATING YOUR FIRST DAG

This Python code defines a simple DAG that simulates a Daily Sales ETL.

Python

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime, timedelta
```

1. Define default arguments

```
default_args = {

    'owner': 'DTU_Student',

    'retries': 3,

    'retry_delay': timedelta(minutes=5),

}
```

2. Initialize the DAG

```
with DAG(

    dag_id='daily_sales_pipeline_v1',

    default_args=default_args,

    start_date=datetime(2026, 3, 1),

    schedule_interval='@daily',

    catchup=False

) as dag:

    def extract(): print("Extracting data from SQL...")

    def transform(): print("Cleaning data with Pandas...")
```

```
def load(): print("Loading data to BigQuery...")
```

3. Define Tasks

```
task_extract = PythonOperator(task_id='extract_data', python_callable=extract)
```

```
task_transform = PythonOperator(task_id='transform_data', python_callable=transform)
```

```
task_load = PythonOperator(task_id='load_data', python_callable=load)
```

4. Set Dependencies

```
task_extract >> task_transform >> task_load
```

VI. ADVANCED FEATURES: XCOMS AND VARIABLES

- **XComs (Cross-Communications):** Allows tasks to exchange small amounts of data (e.g., a task passes a filename to the next task).
- **Variables:** Key-value pairs stored in the Metadata DB (e.g., storing the API endpoint URL) so you don't hardcode sensitive info.
- **Dynamic DAGs:** Using Python loops to generate hundreds of similar DAGs automatically based on a config file.